Velkommen til forelesning #17



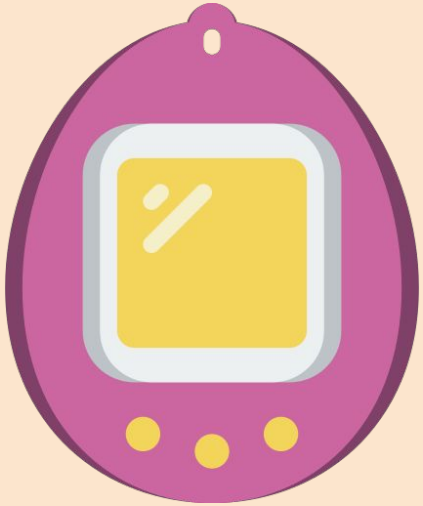

12
9
6
12
9
3
6

Din egen Tamagotchi!

I denne oppgaven skal du lage din egen digitale Tamagotchi!

Du skal lage en liten skapning som du kan mate, leke med og legge til å sove. Skapningen har både glede og energi, og reagerer på det du gjør.

Vi skal bruke **SwiftUI, Model, View og ViewModel**.



Oppgave 1 – Opprett prosjektet

Opprett et nytt iOS-prosjekt i Xcode.

- Kall prosjektet `Tamagotchi`.
- Bruk SwiftUI, men ikke ta med SwiftData
- Opprett tre nye Swift-filer:
  - `Pet.swift`
  - `PetView.swift`
  - `PetViewModel.swift`

Oppgave 2 – Lag en modell

Åpne `Pet.swift` og lag en `struct` som heter `Pet`.

Den skal inneholde fire variabler:

- name – navnet på skapningen (String).
- emoji – hvordan skapningen ser ut (String).
- happiness – hvor glad den er (Int).
- energy – hvor mye energi den har (Int).

Oppgave 3 – Lag en ViewModel

Åpne PetViewModel.swift.

- Lag en klasse som heter `PetViewModel`.
- Bruk `@Observable` slik at SwiftUI kan følge med på endringer.
- Opprett en variabel som heter pet, og gi den en Pet med følgende verdier:
  - Et navn, f.eks “Pip”
  - En emojo, f.eks 🐣
  - Glede: 50
  - Energi: 60

Oppgave 4 – Vis Tamagotchibabyen!

Åpne PetView.swift og lag en SwiftUI-view.

Opprett en @State-variabel som heter viewModel, og gi den en PetViewModel.

Bruk en VStack til å vise:

- Navnet på skapningen.
- En stor emoji.
- Teksten «Ta godt vare på meg!».

Hent navnet og emojien fra ViewModel.

Bruk .font(), .padding() og gjerne litt farger for å lage et hyggelig design.

Husk å vise PetView fra ContentView.

Oppgave 5 – Hvordan har Pip det?

Legg til to ProgressView under skapningen:

- ❤️ Humør
- ⚡ Energi

Hent verdiene fra viewModel.pet.

Begge målerne skal gå fra 0 til 100.

Hint: ProgressView kan vise fremgang basert på en verdi og en totalverdi.
Verdiene må være av typen Double.

Prøv å endre startverdiene i ViewModel. Oppdateres målerne?

Oppgave 6 – Mat, lek og søvn!

Lag tre knapper nederst i appen:

- 🍎 Mat
- 🎾 Lek
- 💤 Sov

Bruk en `HStack` slik at knappene ligger ved siden av hverandre.

Foreløpig skal hver knapp bare skrive en melding til konsollen når du trykker på den.

Oppgave 7 – Gi knappene funksjonalitet

Åpne PetViewModel.swift og lag tre funksjoner.

feed()

- Øker energi med 20.
- Energi kan aldri være høyere enn 100.

play()

- Øker glede med 20.
- Reduserer energi med 10.
- Verdiene kan aldri være lavere enn 0 eller høyere enn 100.

sleep()

- Øker energi med 30.
- Energi kan aldri være høyere enn 100.

Bruk if/else for å kontrollere verdiene.

Koble knappene til funksjonene i ViewModel og test at målerne oppdateres når du trykker!

Oppgave 8 – La skapningen reagere!

Lag en ny String i ViewModel som heter reaction.

Den skal starte med verdien 👋.

Endre reaksjonen når brukeren trykker på knappene:

- Mat: 😋
- Lek: 🥳
- Sov: 😴

Vis reaction under skapningen i PetView.

Test at reaksjonen endres når du trykker på forskjellige knapper.

Oppgave 9 – Hvordan føler Pip seg?

Lag en funksjon i ViewModel som bestemmer hvilken melding skapningen skal vise, basert på hvor glad den er.

Bruk if/else og følgende regler:

- Glede over 70: «Jeg elsker livet! 🥰»
- Glede fra 30 til 70: «Jeg har det ganske fint! 😊»
- Glede under 30: «Kan vi leke litt? 🥺»

Funksjonen skal returnere en String.

Vis resultatet av funksjonen i PetView.

Hint: En funksjon kan returnere en verdi som du kan bruke direkte i en Text.

Oppgave 10 – Gi Tamagotchiene litt liv!

Nå skal vi legge til noen enkle animasjoner.

Del 1 – Animer statusmålerne

Bruk .animation() for å få målerne til å endre seg gradvis når glede eller energi øker og synker.

Prøv animasjonen .easeInOut.

Del 2 – Et sprettende hjerte

- Legg til et hjerte med SF Symbols i PetView.
- Lag en variabel i ViewModel som heter actionCount, med startverdi 0.
- Øk denne med 1 hver gang brukeren trykker på Mat, Lek eller Sov.
- Bruk .symbolEffect() med .bounce for å få hjertet til å sprette når actionCount endres.

Del 3 – Ekstra animasjon

Kan du få selve Tamagotchibabyen til å bevege seg litt?

Eksperimenter gjerne med .scaleEffect() og .animation().

Oppgave 11 – Husk hvordan Pip har det!

Foreløpig glemmer Pip alt når appen avsluttes.

Bruk UserDefaults for å huske hvor glad skapningen er, og hvor mye energi den har.

Del 1 – Lagre verdiene

- Lag en funksjon i ViewModel som heter save().
- Bruk UserDefaults.standard til å lagre happiness og energy.
- Gi begge verdiene hver sin nøkkel.
- Kall save() når skapningens verdier endres.

Del 2 – Hent verdiene

- Bruk init() i ViewModel til å hente lagrede verdier.
- Hvis det finnes lagrede verdier, skal disse brukes.
- Hvis ikke, beholder vi startverdiene fra oppgave 3.

Del 3 – Test

- Start appen og mat Pip noen ganger.
- Avslutt appen helt.
- Åpne appen på nytt.

Har Pip beholdt energien og gleden sin?

⭐ Bonusoppgaver

Bonus 1 – Velg din egen Tamagotchi

Lag et utvalg av forskjellige skapninger som brukeren kan velge mellom.

For eksempel: 🐣 🐱 🐶 🐸 🦊 🐼 🐙 🦄

Bruk gjerne en array og ForEach for å vise alternativene.

Når brukeren velger en skapning, skal emojien oppdateres.

Bonus 2 – Gi skapningen et navn

La brukeren bestemme hva Tamagotchiene skal hete!

- Bruk et TextField hvor navnet kan skrives inn.
- Oppdater navnet på skapningen når brukeren gjør en endring.

Bonus 3 – Tom for energi!

Pip skal ikke kunne leke hvis energien er 0.

- Bruk if/else til å sjekke om skapningen har nok energi.
- Vis en melding hvis Pip er for sliten til å leke.

Bonus 4 – Finn på en ny handling

Kan Tamagotchiene gjøre noe mer?

Kanskje den kan:

- 🚿 Ta en dusj
- 🎵 Danse
- 🍭 Spise godteri
- 🎁 Åpne en gave

Lag en ny knapp og en tilhørende funksjon i ViewModel.

Bestem selv hvordan handlingen påvirker skapningen!

Bonus 5 – Pip savner deg!

Hva om skapningen faktisk savner deg når appen er lukket?

- Bruk Date og UserDefaults til å huske når appen sist var i bruk.
- Når appen åpnes igjen, skal du regne ut hvor lenge den har vært lukket.
- Jo lengre du har vært borte, desto mindre glede skal Pip ha!